

The "Digital Twin" Career Engine

The Objective

Your objective is to build a highly advanced "Digital Twin"—an AI-powered personal manager that analyzes your digital footprint to predict your optimal career path, map your skills, and manage your professional growth. You will use **NotebookLM** for your core knowledge synthesis, **Machine Learning** to predict your future trajectory, and **Antigravity** to both act as your autonomous live agent and "vibe code" your massive web application.

Phase 0: Data Engineering (The Inputs)

If you want a mind-blowing output that actually predicts your future, you need to provide your Digital Twin with high-quality, comprehensive data. Remember that '**Garbage In, Garbage Out**'. You should include *anything* and *everything* that defines your digital, academic, and personal life.

NotebookLM requires clean, structured data. You must format your personal data *at the source* before uploading it to your pipeline.

Upload to NotebookLM as a resources:

1. Your updated CV / Resume (PDF)
2. Your LinkedIn profile
3. Your current Academic Transcript

Optional but welcomed data (The more you include, the better the AI): Do not write complex scripts to extract this data. Copy and paste the raw text into an LLM to format it quickly:

- **AI Chat History:** Extract your technical and soft skills by prompting an LLM with your ChatGPT/Gemini/Perplexity history: *"Export a summary in strict Markdown table format: | Date | Main Topic | Hard Skill Applied | Soft Skill Demonstrated (e.g., Persistence) |"*
 - **YouTube Binge History:** Copy the text of your last 100 watched videos. Paste it into an LLM: *"Format this raw YouTube history into a Markdown table: | Video Title | Category | Technical Relevance |"*
 - **Task/Biometric Data:** Export your task manager (Notion/Trello) or smartwatch data into a clean CSV format.
-

Phase 1: The 5-Layer Gen AI Architecture

1. The Platform Layer (MCP Integration): This is how your system communicates. You must engineer a direct connection between NotebookLM and Antigravity using the **NotebookLM MCP (Model Context Protocol) Server**.

- *Technical Implementation:* Instead of manually exporting files, you will bridge the two platforms so Antigravity can query your NotebookLM data directly.
 - **Install the Server:** Open Antigravity, click the three dots in the top right corner, and select **MCP Servers**. Paste the installation commands for the custom `notebooklm-mcp-server` package directly into the Antigravity chat.
 - **Link the Data:** In NotebookLM, click **Share** and copy the direct link to your notebook.
 - **The Routing Logic:** Go to your Antigravity Settings and change the Auto-Execution review policy to **"Always Proceed"**. Pass the Notebook link in your system prompt, instructing the agent to query the MCP server to extract your soft skills and technical background.
 - *Resource:* Watch this complete tutorial on setting up this exact connection: [Antigravity + NotebookLM is the best VIBE CODING combo | Full tutorial](#)

2. The Model Layer (Reasoning & Prediction): This layer is the brain. Alongside the LLMs, you must vibe code a classical Machine Learning script using `scikit-learn` or a similar framework.

- *Technical Implementation:* 1. **Data Extraction:** Have Antigravity query NotebookLM via MCP to extract an array of your current skills (e.g., `['Python', 'TensorFlow', 'Problem Solving']`). 2. **The Algorithm:** Write a Python script that applies a recommendation or classification algorithm (like **K-Nearest Neighbors (KNN)** or **Cosine Similarity**). 3. **The Training:** Compare your personal "skill vector" against a real-world dataset of tech jobs (you can use a Kaggle dataset containing software engineering job requirements). 4. **The Output:** Your ML script must output a JSON object predicting your top 3 best-fit future jobs, mathematically identifying the "Missing Skills" you need to learn to bridge the gap.

3. The Agent Layer (Live Execution via Antigravity): Give your brain "hands." Build an autonomous Antigravity workflow that takes the career predictions from your Model layer and interacts with the live internet.

- *Technical Implementation:* 1. **System Prompt Injection:** Write a Python script where the Agent's system prompt dynamically accepts the ML output (e.g., `"Your user needs to learn [Missing Skill: WebGL] to become an [AI Interaction Designer]."`) 2. **Tool Calling:** Grant the agent web-browsing capabilities within Antigravity. 3. **Execution:** The agent must autonomously execute a search query, parse the HTML of the top 3 live results, and extract current, real-world resources (like a trending GitHub repo or a live hackathon) to help you learn that specific missing skill.

4. The Application Layer (Vibe-Coded UI): Use Antigravity (or an AI IDE) to "vibe code" a massive, interactive frontend dashboard (e.g., Streamlit, HTML/JS). You must include these UI modules:

- **The Balance Wheel:** A dynamic radar chart visually comparing your Hard Technical Skills versus your Soft Skills.
- **The RPG Tech Tree:** A video-game-style visual map showing the "levels" and skills you need to unlock your ML-predicted dream job.
- **The Live Coach Chatbot:** A chat interface to talk directly to your Digital Twin. You must include a **"Roast My Stack"** toggle button that changes the AI's persona into an aggressive tech lead who humorously critiques your YouTube procrastination and messy code.

- **The "Semester Wrapped" Export:** A beautifully styled, shareable graphic summarizing your semester (e.g., *Top Skill Learned*, *Most Watched Tutorial*, *Most Productive Day*), designed to be posted on LinkedIn.

5. The Infrastructure Layer (Hardware & Hosting): Understand where your compute is happening. You must document which parts of your app run locally on your CPU (like the Streamlit server and ML script) versus which parts run on cloud servers (like the LLM API calls).

Deliverables & Grading

You must submit the following:

1. **The GitHub Repository:** Your complete codebase (ML scripts, Vibe-coded UI, Agent logic). *Do not push your source data to a public repo!*
2. **The Project Demo:** Show us the charts, generate your Semester Wrapped card, trigger the AI Roast Mode, and prove the Agent can browse the web.
3. **The Architecture `README.md` (Crucial for Grading):** Your grade is heavily weighted on this documentation. You must explicitly write out how you engineered each of the **5 Layers** (Platform, Model, Agent, Application, Infrastructure). If you miss a layer in your documentation, your score will reflect it.